ESP Lab1: Signal Modulation by Decimation and Interpolation

1. Learning Outcomes

In this lab, you implement your first multirate system in Matlab and on a Digital-Signal-Processor (DSP) using C programming language. The system to be implemented is shown in Figure 1, and M=L applies.

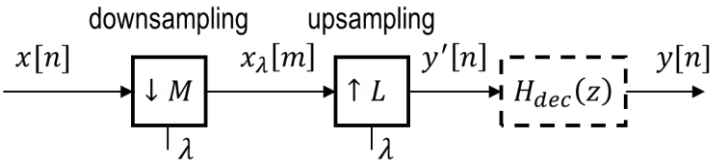


Figure 1: Multirate system to be implemented.

The learning objectives of this lab are to

- understand the properties of downsampling and upsampling in time-domain and frequency-domain,
- know how multirate systems can be used to modulate and demodulate signals,
- be able to convert a solution developed and simulated in Matlab into a C-based solution on a DSP with ADC and DAC interrupt handlers,
- be able to define the requirements for a FIR filter in a multirate filter, design the filter in Matlab, and implement it in C on the DSP,
- know how to use the development environment and its features and configuration of the DSP system,
- know how to use function generator and audio analyzer to debug and understand the DSP system.

2. Task

The conceptual goal is to make bat sounds audible, which normally lie in the range between 40 and 60 kHz and are inaudible to humans. We will mainly work with signals in the range of 15-20 kHz, i.e., in the upper range of human hearing, because this range is definitely supported by standard PC sound cards, and we will initially use chirp signals instead of bat sounds.

Each lab group will be given a slightly different lab assignment and results. Table 1 shows you which parameters to use for which lab group ESPL/01-04. You should use the lab group you selected in myHAW or the one you are scheduled to attend according to your timetable. All teams of a group have the same task.

All results of a laboratory session must be documented in a short report.

You may use generative AI and large language models (LLMs) to complete the tasks, but you **must document the essential prompts used** and be able to explain the source code in detail.

Table 1: Different configurations to be used by different lab groups

Lab group as chosen in myHAW	**Down- / Upsampling factor**	**Chirp, from … to [kHz], length [s]**	**Bat sound** f_{bat}	**Sampling frequency ADC, DAC** f_s [kHz]	**Polyphases** x_λ **output at DAC CH0 / CH1**
ESPL/01	M=L=3	18 … 21 kHz 2.5 s	18 kHz	50 kHz, 50 kHz	$\lambda = 0$ / $\lambda = 2$
ESPL/02	M=L=4	14 … 17 kHz 3.0 s	15 kHz	50 kHz, 50 kHz	$\lambda = 0$ / $\lambda = 3$
ESPL/03	M=L=5	21 … 24 kHz 2.0 s	17 kHz	50 kHz, 50 kHz	$\lambda = 0$ / $\lambda = 4$
ESPL/04	M=L=5	18 … 21 kHz 3.0 s	16 kHz	50 kHz, 50 kHz	$\lambda = 0$ / $\lambda = 3$

3. Preparation

During preparation, you will examine upsampling and downsampling and the related modulation properties in Matlab.

To do this, you will first generate a chirp signal that represents the bat sounds. Then you will examine and compare the time signals and spectrograms before and after downsampling and after upsampling. Initially, neither anti-aliasing filtering nor anti-image filtering will be performed.

In the next step, you will design and implement a suitable anti-image filter and export its coefficients to C for later use in the lab and convert the Matlab script into a C file for the lab.

3.1 Creating a Chirp Signal

Start a new Matlab script and generate a time axis vector and an associated chirp signal $x[n]$ with the parameters given above for your ESPL group. Display the Chirp signal in two subplots as spectrogram $X[l,k]$ and time-domain signal $x[n]$ (stem plot). In the second case, use a signal section ($n = n_1, \dots n_2, n_2 > n_1$) of the chirp signal to make individual sample values clearly visible and distinguishable.

Add your results, including Matlab code and figures, to the preparation sheet. Explain why you think the result is correct.

3.2 Downsampling and Polyphase Representation

Perform undersampling by the factor M given in Table 1 and display the first 100 samples of all M polyphase signals $x_\lambda[m], \lambda = 0, \dots M-1$ **in the domain of the low sampling frequency** f_s' in M subplots one below the other.

Add your results, including Matlab code and figures, to the preparation sheet. Explain why you think the result is correct.

3.3 Upsampling without Interpolation Filter

Now upsample the signal $x_0[m]$ (i.e. $\lambda = 0$) by the factor L (as specified in Table 1 for your ESPL group). Do not apply an interpolation filter at this stage. Display the output signal of the upsampler in two subplots as spectrogram $Y'[l,k]$ and time-domain signal $y'[n]$ (stem plot). In the second case, use a signal section ($n = n_1, \dots n_2, n_2 > n_1$) of the output signal $y'[n]$ to make individual sample values clearly visible and distinguishable.

Add your results, including Matlab code and figures, to the preparation sheet. Explain the appearance of the spectrogram and provide reasons based on specific frequencies. Assume

that a bat sound of f_{bat} (as given in Table 1 for your ESPL group) is fed into the system's ADC. Is it reproduced at the DAC output? If so, at what frequency / frequencies?

3.4 Applying an Interpolation Filter

Design an interpolation filter with $d_{pass} = 0.001$ and $d_{stop} = 0.001(-60\text{dB})$ and a transition width $f_{stop} - f_{pass} = 1\,\text{kHz}$. To do this, use the Matlab command `firpmord()`, which determines the filter order of the Parks-McClellan optimal FIR filter, and then design the filter with `firpm()`. The frequencies f_{pass} , f_{stop} depend on L and must be determined by you. Apply the filter to the output of the upsampler. The Matlab script `Tutorial2_FIR_filter.m` in Moodle will help you use these functions.

Display the filter output signal in two subplots as spectrogram $Y[l,k]$ and time-domain signal $y[n]$ (stem plot). In the second case, use a signal section $(n = n_1, \ldots n_2, n_2 > n_1)$ of the output signal to make individual sample values clearly visible and distinguishable.

Add your results, including Matlab code and figures, to the preparation sheet. Compare the results from 3.3 and 3.4 and explain them.

3.5 Export Filter Coefficients to C

The Matlab script `Tutorial2_FIR_filter.m` uses the `write_coeff()` function to export the filter coefficients to a header file (.h) for your C project later used in the lab. The header file declares the number of filter coefficients, the delay line of the FIR filter (storage elements), and the filter coefficients themselves in 16-bit format.

Generate the header file for the filter coefficients of your interpolation filter and bring the file with you to the lab session. How many coefficients does your FIR filter have? Which is the largest and which the smallest value? Do these values fit into 16-bit format? Explain.

3.6 C Program with Interrupt Handlers

ADC and DAC trigger an interrupt when data has been received or needs to be sent, respectively. The exact timing of the interrupt handlers, i.e., which one occurs first, cannot be predicted.

Decimation is implemented in the ADC handler `adcInt()`and interpolation (and a possible interpolation filter) in the DAC handler `dacInt()`. Data exchange between the interrupt handlers takes place via global variables.

The file `dec_int_standalone.c` implements decimation and interpolation for M=L=2. Compile it using a compiler of your choice (e.g., the campus license for Microsoft Visual Studio or online at https://onecompiler.com/c) and then execute it.

Extend the code for M and L values given in Table 1 for your ESPL group. The polyphases $x_\lambda[m]$ specified in Table 1 should be fed to the respective DAC outputs. Why is `g_PolyWUpdate` necessary? Add your C code to the preparation sheet.

The preparation is complete. Congratulations! **Please submit the preparation sheet in Moodle before the lab session.**

4. Lab task

In the lab, you will implement the multirate system on the DSP and use the audio analyzer to check that it is functioning correctly and has the correct properties.

4.1 Echo program

As described in Section 2 of the document "`Getting Started with UniDAQ2`" we connect the hardware.

Then we import the `adda_simple` project as described in Section 2.1 of the same document. The program forwards the two input channels of the ADC to the two output channels of the DAC ("echo").

Turn on the UPV audio analyzer and select a setup file that can be used to measure the response of our DSP system to a chirp signal across the entire frequency range from (almost) $0\ \mathrm{Hz}$ to $f_s/2 = 25\ \mathrm{kHz}$. We use the audio analyzer here to record the so-called amplitude response or frequency response.

Take a screenshot of the amplitude response and attach it to the lab report. In the report, describe the shape of the frequency response and explain how this affects signal transmission in the DSP system.

4.2 Implementation of multirate system

Take the modified C code prepared in Section 3.6 and insert the necessary parts into the `adda_simple` project. Note that with each ADC interrupt one value on each of the two channels is received and that each DAC interrupt sends one value per channel.

After successful testing, insert the final C code of adcInt() and dacInt() into the lab report. Apply a sine wave of frequency f_{bat} as specified in Table 1 for your group to both inputs of the system and measure the amplitude spectrum via FFT at the output. Take a screenshot for the report. Increase the frequency by $5\ \mathrm{kHz}$ and compare the amplitude spectrum to the previous. Explain your observations.

4.3 Insert the interpolation filter

The interpolation filter that you designed in the prep task in Section 3.4 is now to be inserted into DAC channel 0. To do this, insert the assembly code `FIR_filter_asm_sc.asm` and the prototype definition into your project.

```
// prototype Filter

int16_t FIR_filter_sc(int16_t FIR_delays[],         // delay array
                      int16_t FIR_coe[],            // coefficients
                      int16_t Ndelays,             // number of coeffs / delays
                      int16_t x_n,                  // current input sample
                      int16_t shft);                // use shft=15 as default parameter
```

Afterwards include the header file generated using the Matlab script `write_coeff` in Section 3.5. Add the function call of the filter to the `intDAC()` interrupt handler and run the multirate filter again.

Compare amplitude spectrum and time-domain signals of both DAC channels 0 and 1 again for the bat frequency f_{bat} as given in the Table 1. Add relevant screenshots to your report and explain the results.

4.4 Optional task: let the bats fly

A wave file containing bat sounds has been stored on the internship computer. These are not the original sounds, which are in the 40-50 kHz range, but were shifted to the upper human hearing range. Input the signal to your system and compare the animal sounds at input and output of the multirate system and with and without interpolation filters by listening and using the audio analyzer.

Congratulations! **You made it. Please submit with 15 min. after end of the session.**